Rust: Segurança na Programação

Nulls e Erros

Nuno Neto · 2026

CONNECT5
EDIH European Digital Innovation Hubs Network
REPÚBLICA PORTUGUESA
Financiado pela União Europeia NextGenerationEU
PRR Plano de Recuperação e Resiliência

O que são e por que existem?

Um objeto **null** representa a **ausência de valor**. Introduzido inicialmente na linguagem C para definir ponteiros sem um destino de memória válido.

Necessidade Histórica

Em 1965, a memória era um recurso extremamente escasso. O *null* foi concebido como uma forma eficiente de inicializar ponteiros para indicar "sem referência" sem desperdiçar endereços de memória, garantindo flexibilidade em estruturas de dados complexas.

A Propagação

Devido à adoção massiva do C, o conceito de *null* tornou-se um padrão fundamental em linguagens como C++, Java e Python, servindo como uma ferramenta universal para representar a falta de dados ou estados de erro iniciais.

"The billion dollar mistake" — Tony Hoare, vencedor do prémio Turing em 1980, criador da referência nula em ALGOL W (1965), descreveu sua criação como um "*billion dollar mistake*" devido aos incalculáveis custos de debugging e falhas em sistemas de produção ao longo de décadas.

O que trouxeram de mau?

Null Pointer Exceptions

Um dos erros de *runtime* mais comuns, quase sempre irrecuperáveis.

Complexidade Lógica

Obriga a *null checks* constantes para lidar com valores potencialmente nulos.

Ambiguidade

Um valor nulo não dá informação — foi intencional ou resultado de um erro?

Fraqueza de Tipos

Nulos podem ser atribuídos a qualquer tipo, enfraquecendo o sistema de tipos.

Tentativas de Mitigação

A indústria tem evoluído para substituir o *null* por construções mais seguras:

Tipagem Segura

- **Option/Maybe Types:** Introduzidos em linguagens como Rust, Swift e Haskell para forçar o tratamento explícito de valores ausentes.

Segurança de Compilação

- **Nullable Types:** Linguagens modernas (Kotlin, TypeScript) obrigam a verificação de nulos em tempo de compilação, eliminando a surpresa em tempo de execução.

O tipo Option

Em Rust, todas as variáveis têm de estar **sempre definidas**. Para valores opcionais, usa-se o tipo **Option** — um enum com dois estados:

Some(T)	None
Existe um valor, contido na variável.	Nenhum valor existe — equivalente a nulo.

Vantagens do Option vs. null simples

Segurança Estática

O compilador assegura que todos os casos estão cobertos.

Explicitude

Sabemos sempre quando um valor pode ser nulo e somos obrigados a considerar isso. Distingue nulo intencional de **erro**.

Pattern Matching

Integra-se na perfeição com **match** e **if let**.

Funções de Ordem Superior

Suporta *map*, *zip*, *flatten*, *filter*, *as_ref*, *as_mut*, entre outros.

&Option vs Option<&T>

Foo / Argumento	Option<&T>	&Option<T>
Option<T>	foo.as_ref()	&foo
Option<&T>	foo	Impossível
T	Some(&foo)	&Some(foo)
&T	Some(foo)	Impossível

Option<&T> é sempre preferível: graças à otimização *niche pointer*, ocupa menos memória, requer apenas uma comparação em *match*, e é compatível com qualquer argumento possível.

&Option cria cenários impossíveis de resolver em vários casos e não permite a mesma otimização.

Regra: optar **sempre** por **Option<&T>**.

Erros na Programação

O tratamento de erros robusto assenta em três conceitos fundamentais que garantem a fiabilidade do software:

1. **Informação**
   Identificar e notificar a ocorrência de um estado inesperado ou inválido.
2. **Propagação**
   Encaminhar o contexto do erro através da pilha de chamadas.
3. **Recuperação**
   Intervir para restaurar um estado consistente e funcional.

Detalhes dos Conceitos

Informação

Um erro sem contexto é inútil. Exemplo: lançar uma exceção com uma mensagem clara ("Arquivo não encontrado") em vez de um código genérico.

Propagação

Decidir se a função atual pode resolver o problema ou se deve delegar. Exemplo: subir uma exceção para o controlador da interface.

Recuperação

Ação corretiva. Exemplo: tentar reconectar ao banco de dados ou utilizar um valor padrão (default) caso um recurso falhe.

Estratégias de Tratamento

Exceções (try/catch)

Prós: Separa lógica de negócio do tratamento. **Contras:** Pode esconder fluxos de controle e tornar o código imprevisível.

Result Types (Result<T, E>)

Prós: Força a consideração do erro em tempo de compilação. **Contras:** Pode gerar verbosidade no código (boilerplate).

Por que os três passos são essenciais? Se ignorar a *informação*, o sistema falha silenciosamente. Sem a *propagação*, os erros não chegam a quem pode atuar. Sem a *recuperação*, o sistema colapsa. O equilíbrio entre estes três pontos define a robustez de uma aplicação.

Implicações Práticas

- **Princípio da Fail-Fast:** Detectar erros o mais cedo possível para evitar corrupção de dados.
- **Transparência:** O programador deve saber sempre quais erros uma função pode propagar.
- **Consistência:** Manter a consistência do estado do programa mesmo após o erro (evitar *leaks* de recursos).

Erros em C, C++ e Java

C

Usa **error codes** (inteiros) e a variável global **errno**. Simples, fácil de ignorar, sem mecanismos de propagação ou recuperação embutidos.

C++

Introduz **exceptions** com *try/catch/throw*. Suporta *stack unwinding* e separação de código, mas pode introduzir *overhead* e dificultar a manutenção.

Java

Hierarquia de exceções: **checked** (declaradas na assinatura), **unchecked** (RuntimeExceptions) e **erros** irrecuperáveis. Adiciona também o operador **finally**.

O tipo Result<T, E>

Diferente de linguagens com exceções, Rust trata erros como **parte explícita do sistema de tipos**. Não há fluxo de controle oculto; o compilador garante que erros sejam tratados.

Recuperáveis (Result<T, E>)

Um *enum* que representa sucesso **Ok(T)** ou erro **Err(E)**. *T* é o tipo do sucesso e *E* é o tipo do erro.

Irrecuperáveis (panic!)

Terminam o programa imediatamente. Devem ser evitados em produção, sendo usados apenas para estados lógicos impossíveis.

Por que o Result é superior?

Vantagens do modelo Rust:

- **Obrigatoriedade:** O programador é forçado a lidar com o erro em casos que possam acontecer.
- **Transparência:** Sem saltos invisíveis no código (control flow).
- **Explicitabilidade:** A assinatura da função documenta a possibilidade de falha.

Propagação Explícita:

Ao contrário de exceções que "sobem" automaticamente, a propagação em Rust exige ação (ex: operador ?). Isso torna o custo de cada operação claro e previsível.

Manipulação e Transformação

Processamento

Use match, if let, ou combinadores funcionais como map, map_err, e or para processar valores de forma fluida.

Conversão para Option

É possível converter Result em Option utilizando .ok() (para *T*) ou .err() (para *E*), descartando a outra parte do enum.

Métodos como **unwrap** e **expect** disparam *panic* internamente. Utilize-os apenas em situações onde a falha é logicamente impossível.

Propagação de Erros e o Operador ?

Em Rust, a propagação não é automática. Para evitar a verbosidade de estruturas *match* aninhadas, utilizamos o operador **?**, que simplifica drasticamente o tratamento de erros.

O que é o operador ?

O operador **?** atua como um atalho para a propagação de erros. Ele é colocado após uma expressão que retorna Result<T, E> ou Option<T>.

Como ele funciona

- **Em caso de** Ok(val)**:** O valor é extraído e a expressão continua.
- **Em caso de** Err(e)**:** O erro é retornado imediatamente pela função atual, encerrando o fluxo.
- **Compatibilidade:** Funciona de forma análoga para Option (retorna None em caso de erro).

Comparação: Match vs Operador ?

Com match (Verboso)

```
let f = match file.open() {
    Ok(file) => file,
    Err(e) => return Err(e),
};
```

Com ? (Conciso)

```
let f = file.open()?;
```

Limitação Importante: O operador ? só pode ser utilizado dentro de funções que possuem um tipo de retorno compatível (ex: Result ou Option), pois ele realiza um return antecipado do erro.

Use o ? sempre que desejar propagar o erro para quem chamou a função, mantendo o código limpo e legível.

ERROS EM RUST

Definir Erros e Crates Úteis

Boas Práticas

Tipos de erro devem:

- Implementar **Error** da *std*
- Conter informação acessível programaticamente
- Derivar **Clone** e **Debug**
- Não usar *String* como tipo de erro

thiserror

Criação de tipos de erro com macros. Reduz *boilerplate* e foca na declaração dos tipos. Ideal para **livrarias**.

anyhow

Simplifica a propagação de erros em **aplicações**. Apaga *type information* — não recomendado para livrarias partilhadas.